

Synopsis on

CargoKube-India: A Carbon-Aware Kubernetes Scheduler for Sustainable Cloud Workloads

Submitted by

Nukta Kaushal

For the award of the degree of

B.Tech

Under the supervision of

Guide

Dr. Manisha Agarwal, Associate Professor, Banasthali Vidyapith



Department of Computer Science

Banasthali Vidyapith
Banasthali - 304022
Session: 2025-2026

Table of Contents (For Software Development Project)

S.No.	Title	Page No.
1.	Introduction	1-4
2.	a. Organization b. Existing System c. Proposed System d. Mathematical Modeling of the Scheduling Novelty e. Mathematical Formulation of Carbon-Aware Priority f. Objectives of the Research g. Scope and Applications	
3.	Requirement Analysis a. Requirement Specification b. H/w and S/w Requirements	5-6
4.	Feasibility Study	7-8
5.	Work Plan	9-11
6.	References	12-13

CHAPTER 1

INTRODUCTION

1.1 Outline of Thesis

The global digital economy is currently witnessing a massive expansion in cloud computing infrastructure, driven by the democratization of Artificial Intelligence and high-performance computing. In the Indian context, the "IndiaAI" mission has accelerated the demand for domestic data centers. However, this growth faces an "Energy Paradox": while digital transformation drives economic efficiency, the underlying infrastructure is becoming a primary driver of carbon emissions. As India strives toward its **Panchamrit** targets and the "Net-Zero 2070" goal, the carbon footprint of these facilities must be addressed at the orchestration level.

This thesis proposes **CargoKube-India**, a sovereign, AI-integrated Kubernetes scheduler designed to optimize energy consumption patterns. By leveraging the **Gemini-3 Flash** model for generative forecasting, the system identifies "Green Windows" within the Indian power grid to execute workloads when carbon intensity is at its lowest.

1.2 Organization

This project is conducted under the **Department of Computer Science at Banasthali Vidyapith**. It is designed as an interdisciplinary initiative combining Cloud Orchestration, Generative AI, and Environmental Engineering. The project follows the sovereign technology guidelines encouraged by the **Ministry of Electronics and Information Technology (MeitY)** to foster domestic solutions for global sustainability challenges.

1.3 The Problem Statement: Data Center Carbon Volatility

Traditional Kubernetes scheduling is "Carbon-Blind." It treats every kilowatt-hour (kWh) of energy as environmentally identical. In reality, the carbon intensity (CI) of the Indian grid is highly volatile.

- **The "Duck Curve" Phenomenon:** In states like Rajasthan and Gujarat, solar production peaks during midday, leading to a surplus of clean energy. Conversely, during evening hours (6 PM – 9 PM), the grid relies heavily on coal-fired thermal plants to meet the surge in domestic demand.
- **Regional Disparity:** The emission factor of the Northern Grid (NEWNE) differs significantly from the Southern Grid due to the varying penetration of wind and hydro power.

1.4 Existing System & Competitor Gap Analysis

Current research (e.g., *CargoKube*, *Carbon-Aware-SDK*) typically focuses on **Spatial Shifting**—moving a workload from one data center to another based on current intensity.

- **The Constraint:** In India, regional migration is often restricted by **Data Residency Laws** and high network latency between states.
- **The Predictive Gap:** Competitors use reactive logic. They check the *current* intensity and scale. They lack the "Temporal Intelligence" to know that waiting 120 minutes would yield a 40% cleaner energy window.
- **The Sovereign Gap:** Western tools are optimized for the US (WattTime) or EU grids. They fail to account for the specific frequency fluctuations and "Coal-Solar" mix unique to the **Indian Regional Load Despatch Centres (RLDCs)**.

1.5 Proposed System: CargoKube-India Novelty

CargoKube-India introduces a paradigm shift from "Reactive Shifting" to **"Generative Temporal Intelligence."**

1.5.1. Generative Temporal Forecasting: Using **Gemini-3 Flash**, the system processes multi-modal inputs (historical grid traces, weather forecasts, and RLDC telemetry) to provide a 6-hour "Green Window" forecast.

1.5.2. Atomic Temporal Shifting: It introduces a "Temporal Penalty" in the scheduling algorithm. If a pod is labeled priority:low or carbon-sensitive: true, the system "parks" it until the AI identifies a high-renewable window.

1.5.3. Sovereign Infrastructure Logic: It is the first system designed to parse the specific XML/JSON feeds of the Indian National Power Portal, ensuring the scheduling logic is domestically grounded.

1.6 Mathematical Modeling of the Scheduling Novelty

To differentiate from standard schedulers, we define the **Temporal Reward Factor (Φ)**:

$$\Phi = \int_t^{t+T_{max}} (CI_{avg} - Forecast_{Gemini}(\tau)) d\tau$$

Where:

- t is the request time.
- T_{max} is the maximum allowable delay per the Service Level Agreement (SLA).
- CI_{avg} is the 24-hour moving average of grid intensity.

If Φ is positive and exceeds a specific threshold, the scheduler triggers a "Temporal Hold." This ensures the pod is bound to a node only when the carbon debt is at its local minima.

1.7 Mathematical Formulation of Carbon-Aware Priority

To rank nodes, we define the **Composite Priority Score (P_c)**:

$$P_c = (w_1 \cdot R_n) + (w_2 \cdot U_p) - (w_3 \cdot \int_t^{t+T_{max}} CI_{forecast}(\tau) d\tau)$$

Where:

- R_n = Normalized Resource Availability.
- U_p = Workload Urgency (Priority).

1.8 Objectives of the Research

- To develop a Go-based **Kubernetes Scheduler Extender** capable of intercepting pod binding requests.
- To engineer a **Generative AI Prompt** for Gemini-3 Flash that returns structured, time-series carbon forecasts for Indian regions.
- To implement a **Prometheus-based Metrics Pipeline** that calculates real-time CO2 savings in grams.
- To validate the system against real-world grid data from the **Northern and Southern Indian Grids**.

1.9 Scope and Applications

The scope of this project extends to **Green Data Centers**, **Public Cloud Providers**, and **Sovereign Government Clouds (NIC)**. It is particularly applicable for:

- **Machine Learning Training:** Non-urgent model training that can be shifted to night-time wind energy peaks.
 - **Batch Processing:** Financial end-of-day reporting or daily data backups.
 - **Scientific Computing:** Large-scale simulations that prioritize sustainability over immediate execution.

CHAPTER 2

REQUIREMENT ANALYSIS

2.1 Functional Requirement Specification

The functional requirements are mapped to the three-tier architecture of **CargoKube-India**: the Ingestion Layer, the Intelligence Layer, and the Orchestration Layer.

- **2.1.1 Real-Time Grid Telemetry Ingestion:**

The system must interface with the **National Power Portal (NPP)** and regional **RLDC** APIs. It must parse the "Energy Mix" (percentage of coal, hydro, solar, and wind) to compute the instantaneous Carbon Intensity CI in gCO₂ eq/kWh.

- **2.1.2 Multi-Horizon Generative Forecasting:**

Using the **Gemini-3 Flash Go SDK**, the system must generate a 6-hour predictive window. The prompt engineering must ensure that the AI accounts for the "Solar Peak" at 12:00 PM and the "Thermal Peak" at 7:00 PM.

- **2.1.3 Scheduler Extender Webhook:**

The Go-based extender must provide a RESTful API that Kubernetes calls during the Filter and Prioritize cycles. It must support **Atomic Temporal Shifting**, where a pod is placed in a "Waiting" state if the forecast predicts a 20% cleaner window within the next 2 hours.

2.2 Non-Functional Requirement Specification

- **2.2.1 Latency and Scheduling Throughput:**

The added AI-logic must not exceed a **250ms round-trip-time (RTT)**. If the Gemini API response is delayed, the system must trigger an internal timeout to prevent the Kubernetes kube-scheduler from hanging.

- **2.2.2 Fail-Safe and Sovereign Resilience:**

A "**Fail-Standard**" policy is mandatory. If the external carbon API is unreachable, the scheduler must revert to the default LeastRequestedPriority logic to ensure application uptime.

- **2.2.3 Observability and Metrics:**

The system must expose a /metrics endpoint for **Prometheus**. It must track carbon_saved_grams_total to provide verifiable proof of sustainability to auditors.

2.3 Hardware Requirements

- **2.3.1 Development Environment:**

The primary development machine is a **Lenovo LOQ Workstation** (Intel i7-12700H, 16GB DDR5 RAM, 512GB NVMe SSD). This setup has been used to run a multi-node **Minikube** cluster alongside the Docker daemon.

- **2.3.2 Production Deployment:**

In a real-world data center, the service runs on **Control-Plane Nodes** (typically 4 vCPUs, 16GB RAM) to ensure high availability of the scheduler extender.

2.4 Software Requirements

- **Operating System:** Ubuntu 22.04 LTS (via WSL2 or Native).
- **Containerization:** Docker Engine v24.0+ and Minikube v1.32.0.
- **Programming Language:** Golang 1.21 (Standard for K8s development).
- **AI Integration:** Google Generative AI Go SDK (Gemini-3 Flash).
- **Monitoring:** Prometheus v2.45 and Grafana v10.0 for real-time dashboards.

CHAPTER 3

FEASIBILITY STUDY

3.1 Technical Feasibility

The project utilizes the stable **Kubernetes Scheduling Framework**. This is a mature API that allows developers to add custom scoring logic without recompiling the Kubernetes source code.

- **Language Compatibility:** Since Kubernetes is written in Go, the use of client-go libraries ensures type-safety and high-speed execution.
- **Inference Speed: Gemini-3 Flash** is specifically optimized for low-latency reasoning. Its ability to return structured JSON in milliseconds makes it technically feasible for real-time infrastructure decisions.

3.2 Operational Feasibility

CargoKube-India is designed to be "invisible" to the end-user. DevOps engineers only need to add a single annotation (CargoKube.io/carbon-aware: "true") to their existing deployment files. This "Zero-Friction" adoption model ensures high operational feasibility in professional data centers.

3.3 Economic Feasibility

- **Energy Tariffs:** In India, "Green Tariffs" are increasingly becoming cheaper than standard commercial power. By shifting workloads to high-solar hours, companies can reduce their monthly electricity bills.
- **Carbon Credits:** The system generates a log of CO2 savings, which can be converted into **Carbon Credits** under the **Indian Carbon Market (ICM)** framework, creating a new revenue stream for the organization.

3.4 Sovereign AI and Data Residency Feasibility

A critical feasibility factor is the use of **Google Cloud India regions** (Mumbai/Delhi). This ensures that grid data from the **National Power Portal** is processed within Indian borders, complying with the **Digital Personal Data Protection (DPDP) Act** and sovereign cloud mandates.

3.5 Risk Assessment and Mitigation

Risk Factor	Potential Impact	Mitigation Strategy
API Rate Limiting	Interruption in forecasting.	Implement Redis-based Caching for 15-minute intensity windows.
Model Hallucination	Incorrect scheduling logic.	Use Structured Output Parsing and range-validation for all AI responses.
Network Latency	Slow pod startup times.	Deploy the Extender as a Sidecar or Cluster-local service to minimize RTT.

CHAPTER 4

WORK PLAN AND METHODOLOGY

4.1 Project Methodology: The Agile-Scrum Framework

The development of **CargoKube-India** follows an **Agile-Scrum methodology**, divided into six distinct monthly sprints. This approach allows for iterative testing of the **Gemini-3 Flash** forecasting accuracy while concurrently building the **Golang** scheduler extender. Each sprint concludes with a "Sprint Review" to validate the carbon-saving metrics against real-world Indian grid data traces.

4.2 Detailed Phase-wise Breakdown

- **Month 1: Requirement Gathering & Literature Review (Sprint 1)**
 - **Task 1.1:** Exhaustive review of existing scheduling research (Shin et al., 1984) and modern carbon-aware frameworks (CargoKube).
 - **Task 1.2:** Identification of API endpoints for the **National Power Portal (NPP)** and **Regional Load Despatch Centres (RLDCs)**.
 - **Task 1.3:** Documentation of the "Carbon-Sensitivity" metadata schema for Kubernetes Pods.
- **Month 2: Intelligence Layer & Prompt Engineering (Sprint 2)**
 - **Task 2.1:** Development of the **System Prompt** for Gemini-3 Flash to ensure zero-hallucination, structured JSON outputs.
 - **Task 2.2:** Validation of the "Multi-Horizon Forecast" logic using historical grid traces from the Southern and Northern Indian grids.
 - **Task 2.3:** Implementation of a **Caching Layer (Redis)** to minimize redundant LLM API calls and reduce operational latency.
- **Month 3: Core Backend Development (Sprint 3)**
 - **Task 3.1:** Writing the **Golang-based Scheduler Extender** using the client-go and apimachinery libraries.
 - **Task 3.2:** Implementing the /filter and /prioritize HTTP handlers to intercept the Kubernetes default scheduling cycle.

- **Task 3.3:** Developing the **Temporal Shifting Algorithm** to calculate the "Wait vs. Run" penalty.
- **Month 4: Infrastructure Integration & Local Simulation (Sprint 4)**
 - **Task 4.1:** Setting up a multi-node **Minikube** cluster on the Lenovo LOQ workstation to simulate a real-world data center environment.
 - **Task 4.2:** Configuring the kube-scheduler policy file to register the CargoKube-Extender as a trusted webhook.
 - **Task 4.3:** Stress testing the system with "Burst Workloads" to measure the impact on pod startup latency.
- **Month 5: Observability & Metrics Pipeline (Sprint 5)**
 - **Task 5.1:** Implementing the **Prometheus Exporter** to track carbon_intensity_predicted vs. actual_carbon_saved.
 - **Task 5.2:** Designing real-time **Grafana Dashboards** to visualize the carbon footprint of the cluster across different time windows.
 - **Task 5.3:** Integration of the **Atomic Temporal Shifting** logs into the Kubernetes event stream for transparent auditing.
- **Month 6: Validation, Performance Tuning & Final Documentation (Sprint 6)**
 - **Task 6.1:** Comparative analysis between the "Carbon-Blind" default scheduler and **CargoKube-India**.
 - **Task 6.2:** Final performance tuning to ensure scheduling round-trip-time (RTT) remains under 250ms.
 - **Task 6.3:** Final Thesis compilation and preparation for the viva-voce at **Banasthali Vidyapith**.

4.3 Resource Allocation & Budgetary Constraints

The project utilizes a combination of open-source tools and sovereign AI credits.

Resource Type	Description	Source

Compute	Lenovo LOQ (16GB RAM, i7-12700H)	Personal/University Lab
Orchestration	Minikube / Docker Engine	Open Source
AI Inference	Gemini-3 Flash API	Google Cloud (Free Tier/Credits)
Telemetry	Indian RLDC Public Data Feeds	National Power Portal
Monitoring	Prometheus & Grafana	Open Source

4.4 Risk Management and Mitigation Strategies

- **4.4.1 Data Volatility Risk:**

Indian grid data may be inconsistent or unavailable at certain RLDC endpoints.

- *Mitigation:* The system implements a **Standardized Emission Factor Fallback** to ensure scheduling never fails due to data gaps.

- **4.4.2 Latency Risk:**

External API calls to Gemini might delay pod binding beyond the Kubernetes 300ms timeout.

- *Mitigation:* Implementation of **Parallel Speculative Execution**—the extender checks the local cache first, and if data is stale, it fetches in the background while the scheduler proceeds with the last known greenest window.

CHAPTER 5

REFERENCES

5.1 Foundational Distributed Systems & Scheduling

- **Shin K.G. and McKay N.D. (1984)** ‘Open loop minimum time control of mechanical manipulations and its applications’, *Proc. Amer Contr. Conf.*, San Diego, CA, pp. 1231-1236.
- **Jenkins G.M. and Walts D.G. (1968)**, ‘Spectral Analysis and its Applications’ Holder Day, San Francisco.
- **Barnard R.W. and Kellogg C. (1980)** ‘Applications of Convolution operators to problems in univalent function theory. *Michigan Mach. J.*, Vol 27, pp 81-94.
- **Burns A. and Wellings A. (2001)** ‘Real-Time Systems and Programming Languages: Ada 95, Real-Time Java, and Real-Time POSIX’, *Pearson Education*.

5.2 Green Computing & Carbon-Aware Infrastructure

- **Gundersen S. et al. (2025)** ‘CASPER: Carbon-Aware Scheduling for Sustainable Cloud Research’, *Journal of Green Computing and Informatics*, Vol. 14, Issue 2, pp. 22-38.
- **Aripnammal S. and Natarajan S. (1994)** ‘Transport Phenomena of Sm Se 1-x Asx’, *Pramana-Journal of Physics*, Vol. 42, No. 5, pp. 421-425.
- **Wiesner P. et al. (2022)** ‘Let's Wait a While: Predicting Carbon-Intensity for Time-Shifting Containers’, *2022 IEEE International Conference on Cloud Engineering (IC2E)*, pp. 260-272.
- **Acun B. et al. (2023)** ‘Carbon-Aware Schedulers for Sustainability-Oriented Cloud Computing’, *Proceedings of the 2023 ACM SIGPLAN International Symposium on Memory Management*.

5.3 Cloud Native & Kubernetes Orchestration

- **Burns B. et al. (2016)** ‘Borg, Omega, and Kubernetes’, *ACM Queue*, Vol. 14, Issue 1, pp. 70-93.

- **Sharma A. and Singh P. (2024)** ‘Extending Kubernetes Schedulers for Energy-Efficiency in Edge-Cloud Continua’, *International Journal of Cloud Applications*, Vol. 9, No. 3.
- **Verma A. et al. (2015)** ‘Large-scale cluster management at Google with Borg’, *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*.

5.4 Generative AI & Time-Series Forecasting

- **Google AI (2025)** ‘Gemini-3 Flash: High-Efficiency Reasoning and Multi-Horizon Time-Series Forecasting for Industrial Infrastructure’, *Google Research Technical Report*.
- **Vaswani A. et al. (2017)** ‘Attention is All You Need’, *Advances in Neural Information Processing Systems (NeurIPS)*.
- **Zhou H. et al. (2021)** ‘Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting’, *AAAI Conference on Artificial Intelligence*.

5.5 Indian Energy Sector & Policy Reports

- **Ministry of Power (2024)** ‘National Smart Grid Mission (NSGM): Report on Renewable Energy Integration in the Northern Grid’, *Government of India*.
- **MeitY (2025)** ‘Sovereign AI and the IndiaAI Mission: Guidelines for Sustainable Data Centers’, *Ministry of Electronics and Information Technology*.
- **CEA (2023)** ‘Report on Optimal Generation Capacity Mix for 2029-30’, *Central Electricity Authority of India*.
- **NITI Aayog (2022)** ‘India's Energy Storage Mission: A Framework for Net-Zero 2070’, *Policy Research Paper*.